Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Seaborn Notes

Part 1 -- Relational, Distribution & Matrix Plots

Compiled by Bikkad IT Institute | Teaching Notes

Seaborn Notes -- Part 1

Part 1 of 2: A) Relational | B) Distribution | C) Matrix Plots

Axes-level vs Figure-level functions, step by step | Prepared for Bikkad IT Institute Students | Level: Beginner | Jupyter Notebook / Google Colab

Topic Index

0. Setup & Meet the Datasets
1. Axes-level vs Figure-level -- The Real Difference

A) Relational Plots

2. Scatter Plot -- scatterplot() vs relplot(kind='scatter')
3. Line Plot -- lineplot() vs relplot(kind='line')

B) Distribution Plots

4. Histogram -- histplot() vs displot(kind='hist')
5. KDE Plot -- kdeplot() vs displot(kind='kde')
6. Rug Plot -- rugplot() vs displot(rug=True)

C) Matrix Plots

7. Heatmap -- heatmap() (axes-level only)
8. Cluster Map -- clustermap() (figure-level only)

9. Quick Revision Cheat Sheet & Practice Exercises

How to use these notes: every chart is drawn FIRST with its **axes-level** function (one plot on one axes) and THEN with its **figure-level** function (a whole figure that can be split into panels with col=). Each step is a complete, short cell you can copy-paste and run, followed by the real chart it produces. **Lines in bold are new or changed** compared with the previous step. A small comparison table at the end of every chart sums up the difference.

0. Setup & Meet the Datasets

This part uses 3 small, everyday-life datasets (CSV files, also in one Excel workbook). They are deliberately small so students can look at the whole table and connect every dot or bar to a real row. Import the libraries and read the datasets **once** at the top of the notebook -- every example below reuses these DataFrames. sns.set_theme() gives every chart a clean grid style.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid')

cafe = pd.read_csv('chai_cafe_sales.csv')
marks = pd.read_csv('student_marks.csv')
city = pd.read_csv('city_monthly_temperature.csv', index_col='city')
```

Dataset 1: chai_cafe_sales.csv (variable: cafe)

A chai cafe with two branches (Pune and Nashik) records its daily sales for 2 weeks: temperature, cups of hot chai, cups of cold coffee and the total sales in rupees. Hot days = more cold coffee, less chai; weekends = more customers.

Shape: 28 rows x 7 columns | **Used in:** Scatter Plot, Line Plot | **First 5 rows:**

day	weekday	branch	temperature_c	chai_cups	cold_coffee_cups	total_sales_rs
1	Mon	Pune	29	251	32	7900
2	Tue	Pune	30	244	32	7760
3	Wed	Pune	31	238	38	8180
4	Thu	Pune	33	233	62	10240
5	Fri	Pune	32	234	46	8820

Dataset 2: student_marks.csv (variable: marks)

Marks of 40 students from two classes (10-A and 10-B) in Maths, Science and English, with gender and daily study hours. Class 10-A studies a little more -- will their marks look different?

Shape: 40 rows x 8 columns | **Used in:** Histogram, KDE, Rug Plot, Correlation Heatmap | **First 5 rows:**

roll_no	name	class	gender	study_hours	maths	science	english
1	Aarav	10-A	Male	3.7	62	68	80
2	Ishita	10-A	Female	4.5	82	73	63
3	Rohan	10-A	Male	5.2	75	81	71
4	Sneha	10-A	Female	3.1	50	55	68
5	Kunal	10-A	Male	4.0	72	80	64

Dataset 3: city_monthly_temperature.csv (variable: city)

Typical (approximate) average temperature in deg C of 8 Indian cities for every month. Cities are the rows and months are the columns -- exactly the shape a heatmap needs. Hill stations (Shimla, Darjeeling) are much cooler than the plains.

Shape: 8 rows x 12 columns (a ready-made matrix) | **Used in:** Heatmap, Cluster Map | **First 5 rows (Jan-Aug shown):**

city	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug
Pune	21	23	27	30	30	27	25	24
Mumbai	24	25	27	29	30	29	28	27
Chennai	25	26	28	31	33	32	31	30
Kolkata	20	23	28	31	31	30	29	29
Delhi	14	17	23	30	34	34	31	30

Teaching Note

Before drawing anything, let students run .head(), .shape and .describe() on each dataset and say in their own words what one row means. Seaborn always works the same way: data= the DataFrame, and x= / y= / hue= the column NAMES.

1. Axes-level vs Figure-level -- The Real Difference

Seaborn has two kinds of plotting functions. An **axes-level** function (scatterplot, histplot, boxplot ...) draws ONE plot on ONE Matplotlib axes -- you control the figure with plt. A **figure-level** function (relplot, displot, catplot ...) creates the WHOLE figure by itself and can split the data into several panels with col= and row=. Same chart, two ways to draw it. The four steps below show the difference with real output.

Dataset: student_marks.csv

Syntax

```
ax = sns.histplot(data=df, x='col')        # axes-level
g  = sns.displot(data=df, x='col', kind='hist')  # figure-level
fig, axes = plt.subplots(1, 2)   +   ax=axes[0]
sns.displot(..., col='col_name')  # one panel per value
```

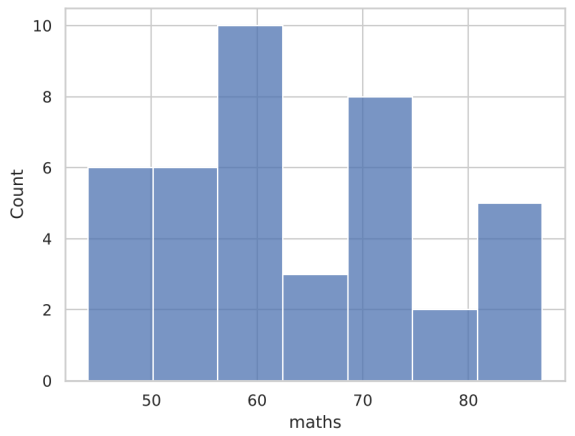
1.1 Axes-level function returns an Axes

```
ax = sns.histplot(data=marks, x='maths')
print(type(ax))
```

Output / Result

```
<class 'matplotlib.axes._axes.Axes'>
```

Output of step 1.1:



Explanation

sns.histplot() draws on the current Matplotlib axes and gives that axes back. So everything you learnt in Matplotlib (plt.title(), plt.xlabel(), plt.figure(figsize=)) works with it.

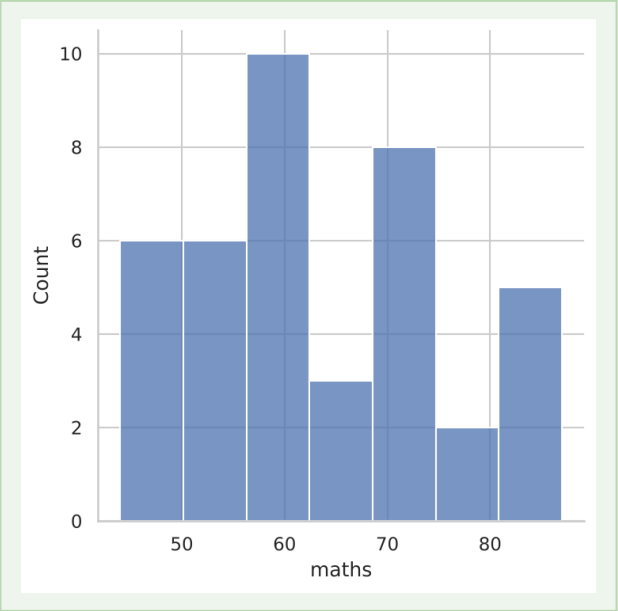
1.2 Figure-level function returns a FacetGrid

```
g = sns.displot(data=marks, x='maths', kind='hist')
print(type(g))
```

Output / Result

```
<class 'seaborn.axisgrid.FacetGrid'>
```

Output of step 1.2:



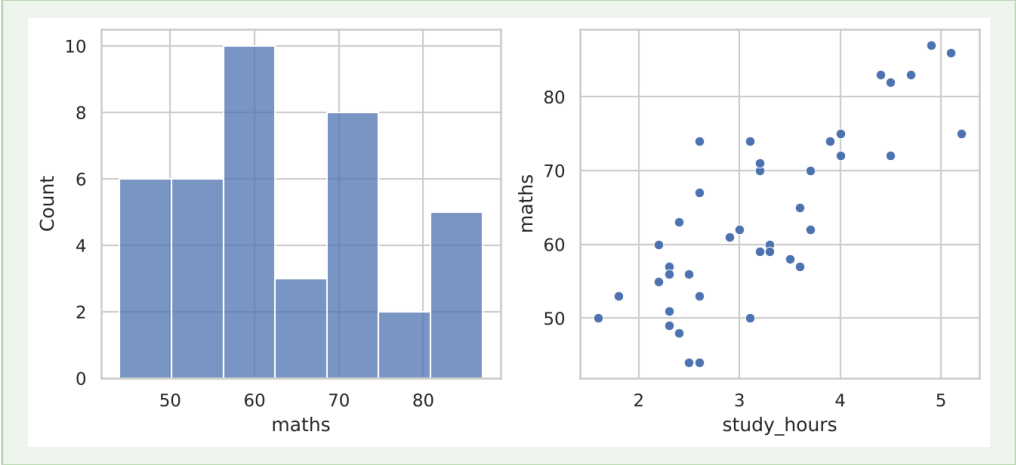
Explanation

sns.displot() draws the same histogram, but it builds its own figure and returns a Seaborn **FacetGrid** object (a grid of panels). The chart looks almost the same -- the difference is what you can do with it next.

1.3 Axes-level: different plots side by side with ax=

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.histplot(data=marks, x='maths', ax=axes[0])
sns.scatterplot(data=marks, x='study_hours', y='maths', ax=axes[1])
```

Output of step 1.3:



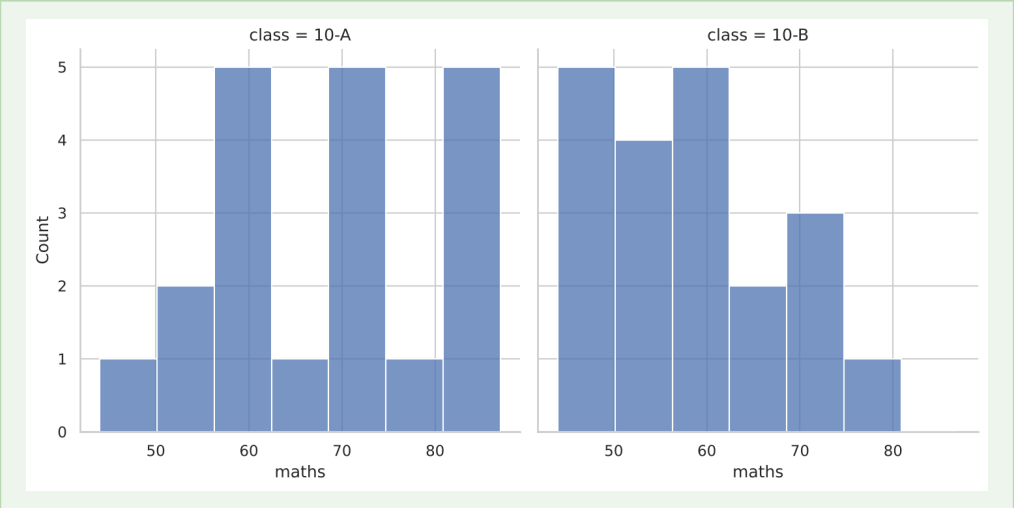
Explanation

Because axes-level functions accept ax=, you can put DIFFERENT charts into your own plt.subplots() layout -- a histogram on the left and a scatter plot on the right. Figure-level functions cannot do this (they have no ax= argument).

1.4 Figure-level: one panel per group with col=

```
sns.displot(data=marks, x='maths', kind='hist', col='class')
```

Output of step 1.4:



Explanation

With col='class', displot() automatically makes one panel for 10-A and one for 10-B, with the same axis scale, so they are easy to compare. With an axes-level function you would have to filter the data and draw each panel yourself.

Teaching Note

Easy way to remember: axes-level = ONE chart that you place yourself (ax=). Figure-level = a whole FIGURE that Seaborn builds for you, and can split into panels (col=, row=). In this notes, every chart is drawn first with its axes-level function and then with its figure-level function.

A) Relational Plots

2. Scatter Plot -- scatterplot() vs relplot(kind='scatter')

A scatter plot shows the **relationship between two numeric columns** -- one dot per row. Use case: does one number go up when the other goes up? Here we ask: on hotter days, does the cafe sell more cold coffee?

Dataset: chai_cafe_sales.csv

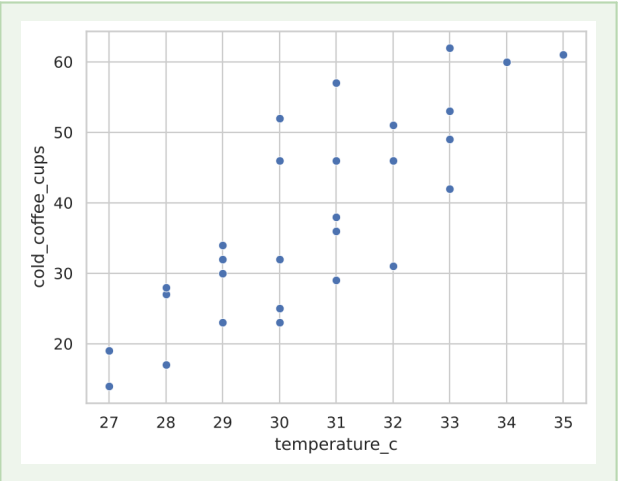
Syntax

```
sns.scatterplot(data=df, x='col', y='col', hue=, style=, size=)
sns.relplot(data=df, x='col', y='col', kind='scatter',
            hue=, col=, height=, aspect=)
```

2.1 Axes-level: a plain scatter plot

```
sns.scatterplot(data=cafe, x='temperature_c', y='cold_coffee_cups')
```

Output of step 2.1:



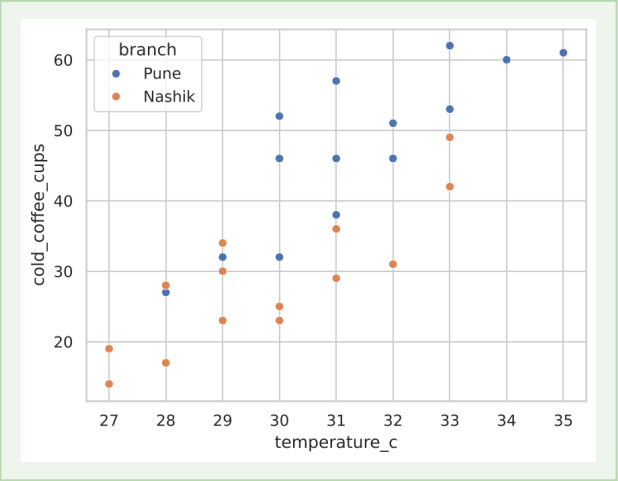
Explanation

data= is the DataFrame and x= / y= are column NAMES (in quotes). Seaborn reads the columns, draws 28 dots and even writes the axis labels for us. The dots go up to the right: hotter day, more cold coffee.

2.2 hue= -- colour by a category

```
sns.scatterplot(data=cafe, x='temperature_c', y='cold_coffee_cups',
                hue='branch')
```

Output of step 2.2:



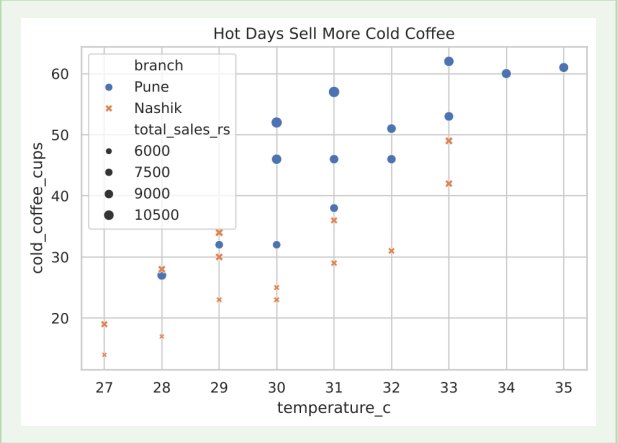
Explanation

hue='branch' gives each branch its own colour and adds a legend automatically -- in Matplotlib we needed one plt.scatter() call per group. Pune sells more cold coffee than Nashik at the same temperature.

2.3 style=, size= and plt.title()

```
plt.figure(figsize=(7, 4.5))
sns.scatterplot(data=cafe, x='temperature_c', y='cold_coffee_cups',
                hue='branch', style='branch', size='total_sales_rs')
plt.title('Hot Days Sell More Cold Coffee')
```

Output of step 2.3:



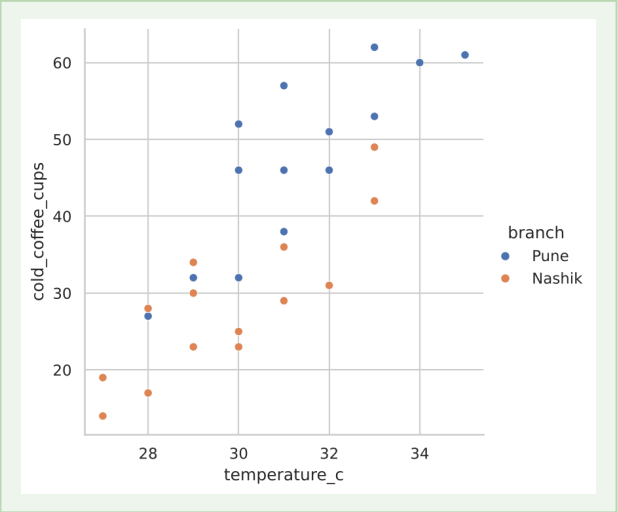
Explanation

style= gives each branch a different marker shape, and size= makes bigger dots for bigger total sales. Because this is an axes-level function, plt.figure(figsize=) and plt.title() work exactly as in Matplotlib.

2.4 Figure-level: relplot(kind='scatter')

```
sns.relplot(data=cafe, x='temperature_c', y='cold_coffee_cups',
            hue='branch', kind='scatter')
```

Output of step 2.4:



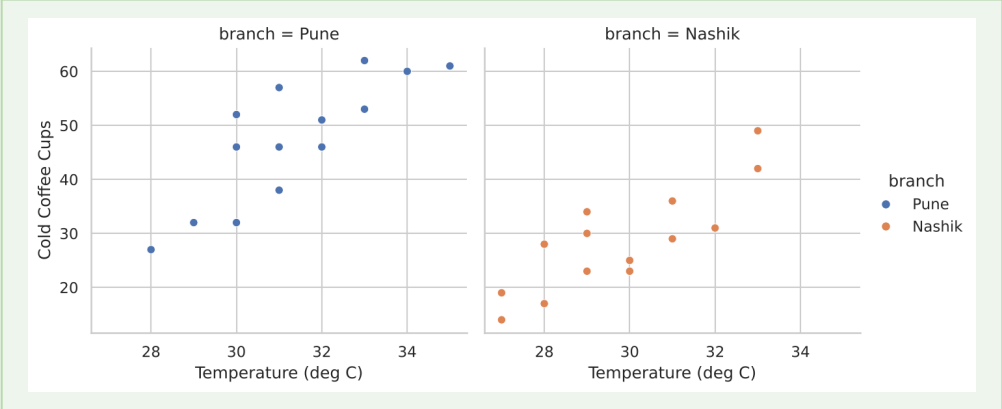
Explanation

The same chart with the figure-level function. Note the legend is now placed OUTSIDE the plot on the right -- figure-level functions always do this. kind='scatter' is the default for relplot().

2.5 Figure-level: col= makes one panel per branch

```
g = sns.relplot(data=cafe, x='temperature_c', y='cold_coffee_cups',
                hue='branch', kind='scatter', col='branch',
                height=4, aspect=1.1)
g.set_axis_labels('Temperature (deg C)', 'Cold Coffee Cups')
```

Output of step 2.5:



Explanation

col='branch' splits the data into a Pune panel and a Nashik panel. The size is set with height= (inches per panel) and aspect= (width = height x aspect), NOT figsize. Labels for all panels are set at once with g.set_axis_labels().

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.scatterplot()`	`sns.relplot(kind='scatter')`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`Make them yourself with plt.subplots()`	`col= / row=`
Labels	`plt.title(), plt.xlabel()`	`g.set_axis_labels()`

Teaching Note
Ask students: which branch sells more cold coffee on a 31 deg C day? Then let them use y='chai_cups' -- chai sales tend to go DOWN on hot days (a weak negative relationship).

3. Line Plot -- lineplot() vs relplot(kind='line')

A line plot joins the points in **order**, so it is used for **trends over time** -- days, months, years. A special Seaborn feature: when one x value has many rows, lineplot() draws their **average** and a shaded confidence band automatically. Our question: how do daily sales change through the two weeks?

Dataset: chai_cafe_sales.csv

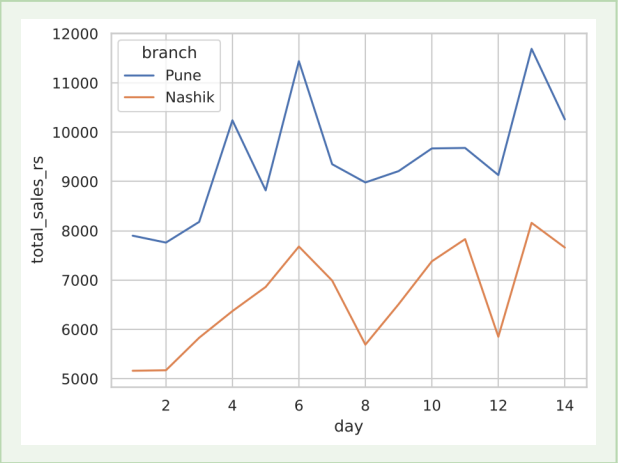
Syntax

```
sns.lineplot(data=df, x='time', y='num', hue=, marker=,
             errorbar=)
sns.relplot(data=df, x='time', y='num', kind='line',
            hue=, col=, height=, aspect=)
```

3.1 Axes-level: one line per branch with hue=

```
sns.lineplot(data=cafe, x='day', y='total_sales_rs', hue='branch')
```

Output of step 3.1:



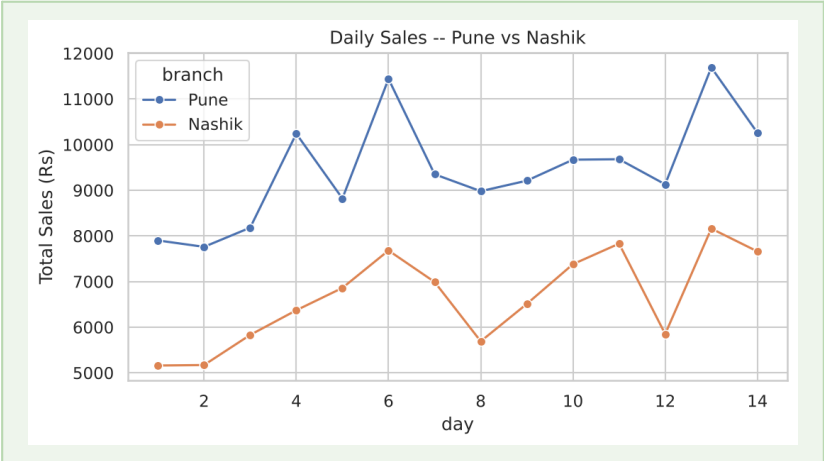
Explanation

x='day' runs 1 to 14 and hue='branch' draws a separate line for each branch. Both lines jump on days 6-7 and 13-14 -- the weekends.

3.2 marker= and plt.title()

```
plt.figure(figsize=(8, 4))
sns.lineplot(data=cafe, x='day', y='total_sales_rs', hue='branch',
             marker='o')
plt.title('Daily Sales -- Pune vs Nashik')
plt.ylabel('Total Sales (Rs)')
```

Output of step 3.2:



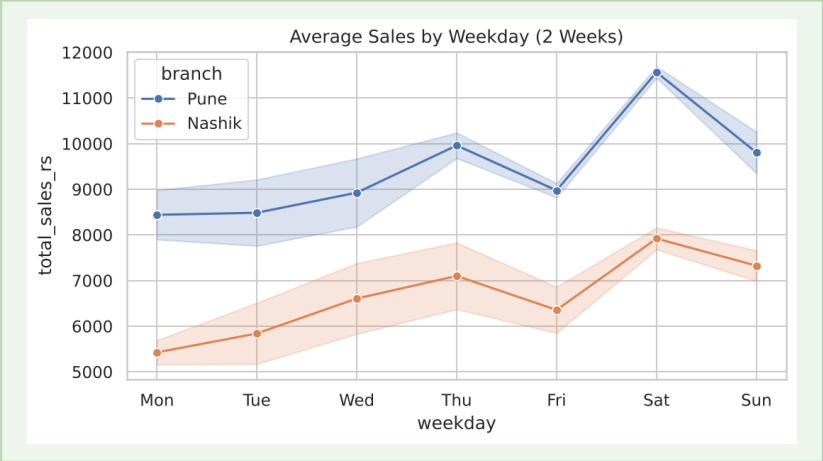
Explanation

marker='o' puts a dot on every real reading. A wider figure and a title make it ready for a report -- all normal Matplotlib functions because lineplot() is axes-level.

3.3 Automatic average + confidence band

```
plt.figure(figsize=(8, 4))
sns.lineplot(data=cafe, x='weekday', y='total_sales_rs', hue='branch',
             marker='o')
plt.title('Average Sales by Weekday (2 Weeks)')
```

Output of step 3.3:



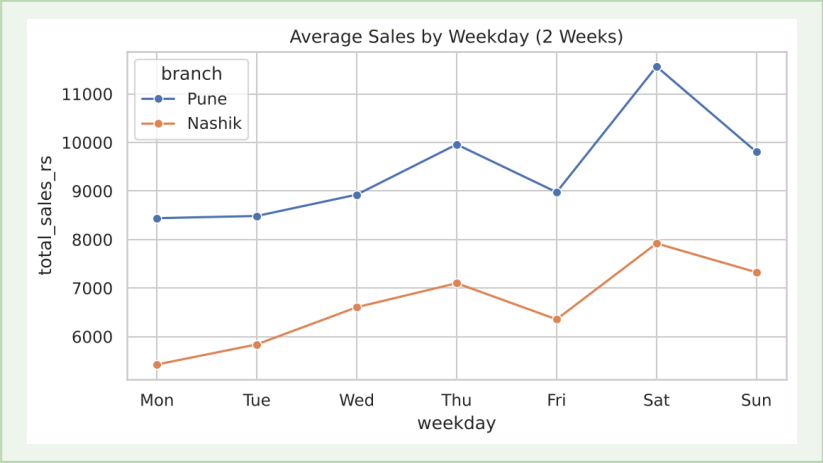
Explanation

Now x='weekday': each weekday appears TWICE (week 1 and week 2). Seaborn plots the AVERAGE of the two values and shades a band showing how much they varied. Matplotlib would need a groupby() first -- Seaborn does it for you.

3.4 errorbar=None -- remove the band

```
plt.figure(figsize=(8, 4))
sns.lineplot(data=cafe, x='weekday', y='total_sales_rs', hue='branch',
             marker='o', errorbar=None)
plt.title('Average Sales by Weekday (2 Weeks)')
```

Output of step 3.4:



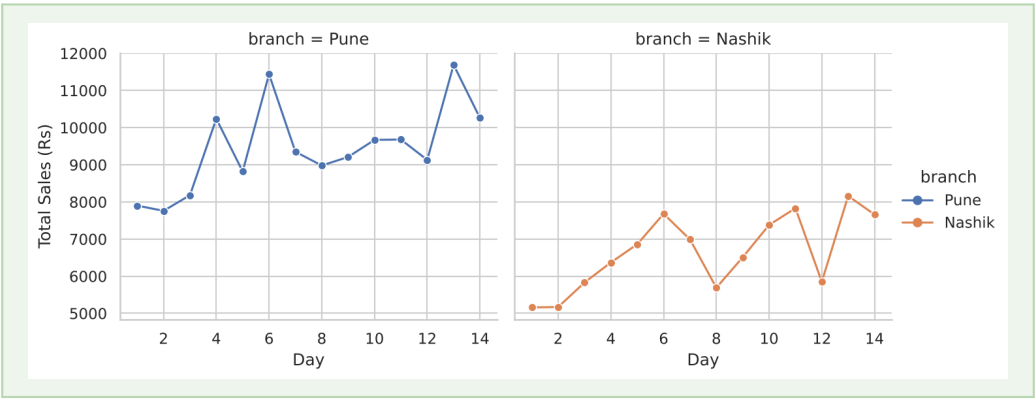
Explanation

errorbar=None keeps the average line but hides the shaded band -- cleaner when the audience only needs the trend.

3.5 Figure-level: relplot(kind='line') with col=

```
g = sns.relplot(data=cafe, x='day', y='total_sales_rs',
                hue='branch', kind='line', marker='o',
                col='branch', height=3.8, aspect=1.2)
g.set_axis_labels('Day', 'Total Sales (Rs)')
```

Output of step 3.5:



Explanation

kind='line' turns relplot() into a line plot. col='branch' gives each branch its own panel with the SAME y-axis scale, so it is fair to compare them. Same arguments as lineplot(), plus col=, height= and aspect=.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.lineplot()`	`sns.relplot(kind='line')`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`
Average + band	`Yes (automatic)`	`Yes (automatic)`

Teaching Note
The scatter plot and line plot share ONE figure-level function: relplot(). Only kind= changes. Show students both cells next to each other on the board.

B) Distribution Plots

4. Histogram -- histplot() vs displot(kind='hist')

A histogram shows the **distribution of ONE numeric column**: it groups values into ranges (bins) and counts how many fall in each. Use case: where do most values lie? Are there very low or very high values? Our question: how are the Maths marks spread?

Dataset: student_marks.csv

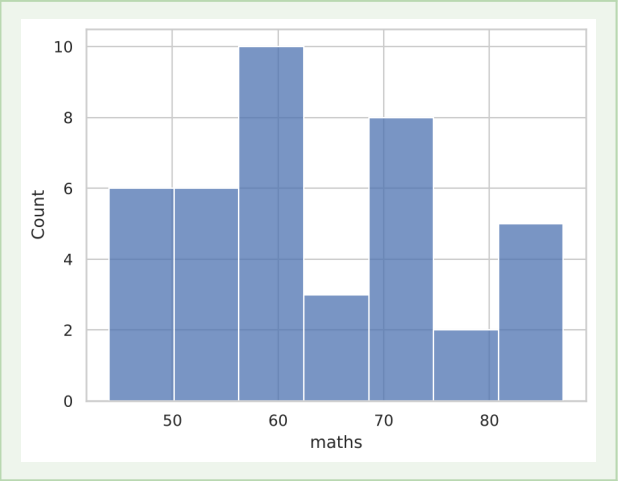
Syntax

```
sns.histplot(data=df, x='num', bins=, hue=, kde=,
             multiple=, color=)
sns.displot(data=df, x='num', kind='hist', bins=, hue=,
            col=, height=, aspect=)
```

4.1 Axes-level: a plain histogram

```
sns.histplot(data=marks, x='maths')
```

Output of step 4.1:



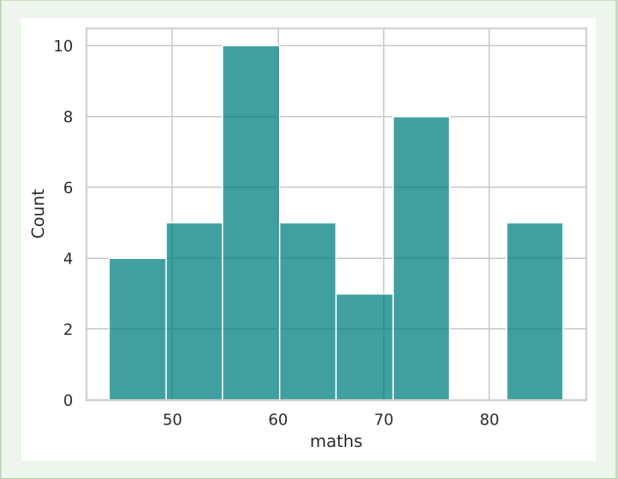
Explanation

Only x= is needed. Seaborn chooses the bins, counts the students in each range and labels the y-axis 'Count'. Most students scored between 50 and 75.

4.2 bins= and color=

```
sns.histplot(data=marks, x='maths', bins=8, color='teal')
```

Output of step 4.2:



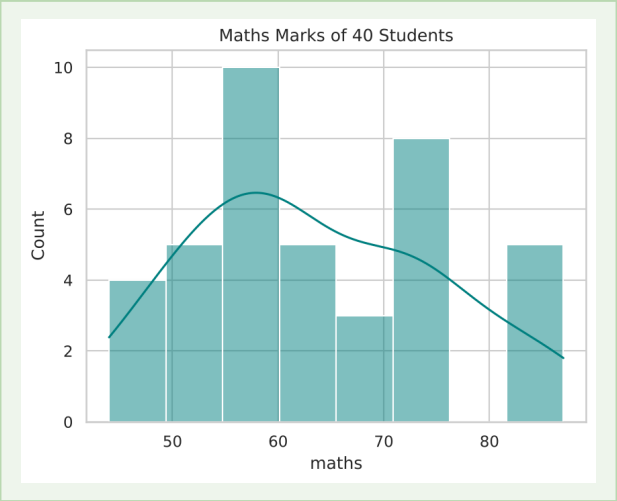
Explanation

bins=8 asks for 8 equal ranges. Try 5, 8 and 15 to see how the shape changes -- too few bins hide detail, too many look noisy.

4.3 kde=True -- add a smooth curve

```
sns.histplot(data=marks, x='maths', bins=8, color='teal',
             kde=True)
plt.title('Maths Marks of 40 Students')
```

Output of step 4.3:



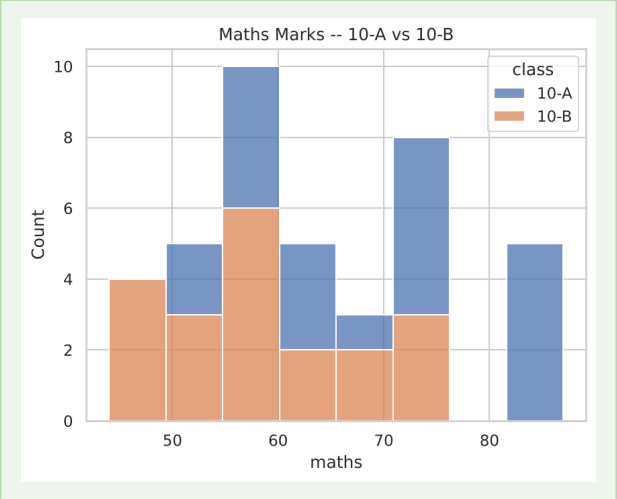
Explanation

kde=True draws a smooth curve over the bars that shows the overall shape of the distribution -- a preview of the KDE plot in the next topic.

4.4 hue= -- compare two classes on the same axes

```
sns.histplot(data=marks, x='maths', bins=8, hue='class',
             multiple='stack')
plt.title('Maths Marks -- 10-A vs 10-B')
```

Output of step 4.4:



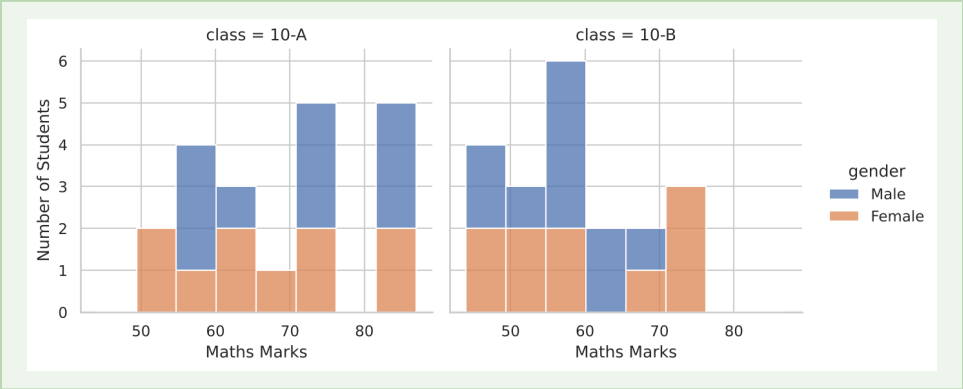
Explanation

hue='class' colours the bars by class. multiple='stack' piles the two colours on top of each other instead of overlapping them. 10-A (more study hours) sits more to the right -- higher marks.

4.5 Figure-level: displot(kind='hist') with col=

```
g = sns.displot(data=marks, x='maths', kind='hist', bins=8,
                col='class', hue='gender', multiple='stack',
                height=3.8, aspect=1.1)
g.set_axis_labels('Maths Marks', 'Number of Students')
```

Output of step 4.5:



Explanation

displot() is the figure-level partner. col='class' makes one panel per class (same x-axis, easy to compare) and hue='gender' splits each panel by gender -- two groupings in one figure.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.histplot()`	`sns.displot(kind='hist')`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Compare groups	`hue=` on one axes	`hue=` and/or `col=` panels
Labels	`plt.title(), plt.xlabel()`	`g.set_axis_labels()`

Teaching Note

Ask students to change x='maths' to x='study_hours' and explain the shape they see. Then ask: which class studies more?

5. KDE Plot -- kdeplot() vs displot(kind='kde')

A KDE (Kernel Density Estimate) plot is a **smooth version of a histogram** -- a curve instead of bars. The higher the curve, the more values are there. Use case: comparing the SHAPE of two or more groups without bars overlapping.

Dataset: student_marks.csv

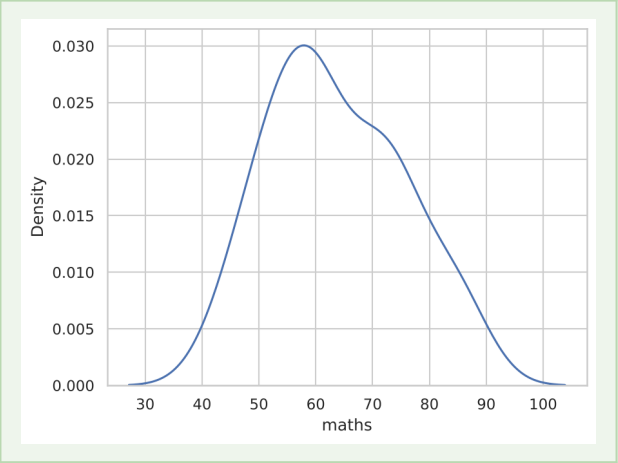
Syntax

```
sns.kdeplot(data=df, x='num', hue=, fill=, alpha=,
            linewidth=)
sns.displot(data=df, x='num', kind='kde', hue=, fill=, col=)
```

5.1 Axes-level: a plain KDE curve

```
sns.kdeplot(data=marks, x='maths')
```

Output of step 5.1:



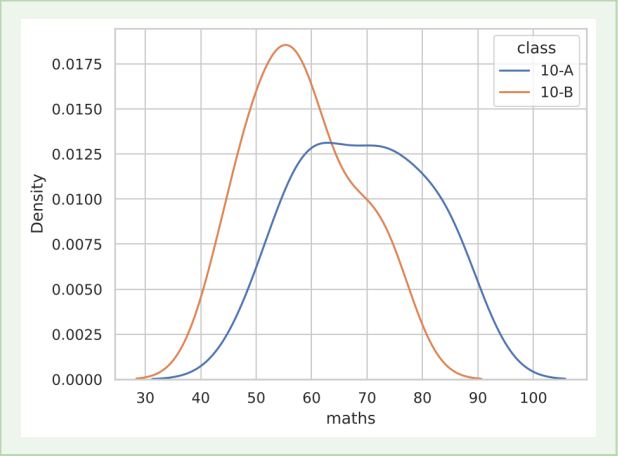
Explanation

One smooth curve for all 40 students. The peak is around 60 -- the most common Maths score. The y-axis is 'Density', not a count.

5.2 hue= -- one curve per class

```
sns.kdeplot(data=marks, x='maths', hue='class')
```

Output of step 5.2:



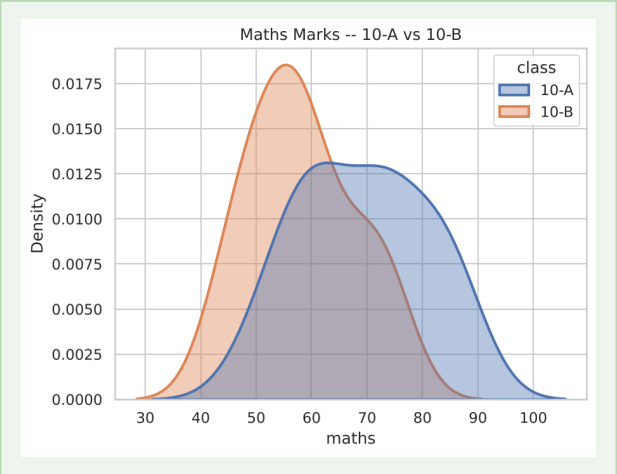
Explanation

Two curves on the same axes: 10-A's peak is further right than 10-B's, so 10-A generally scores higher. Two curves are much easier to compare than two overlapping histograms.

5.3 fill=True and alpha=

```
sns.kdeplot(data=marks, x='maths', hue='class', fill=True,
            alpha=0.4, linewidth=2)
plt.title('Maths Marks -- 10-A vs 10-B')
```

Output of step 5.3:



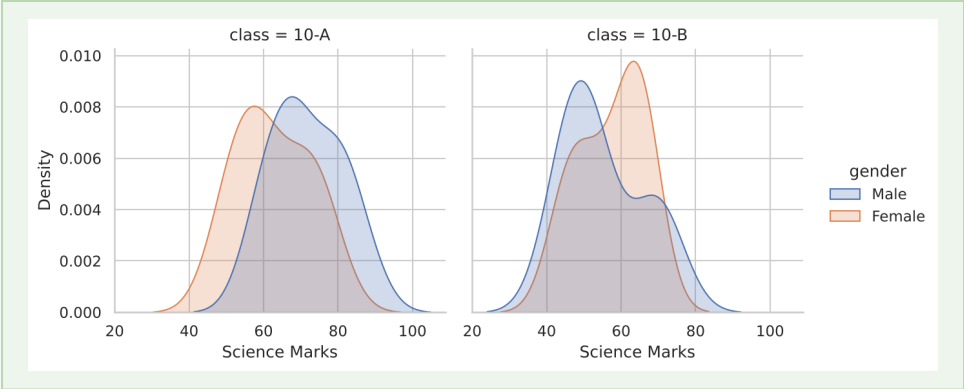
Explanation

fill=True shades the area under each curve and alpha=0.4 keeps it see-through so both shapes stay visible where they overlap.

5.4 Figure-level: displot(kind='kde') with col=

```
g = sns.displot(data=marks, x='science', kind='kde', hue='gender',
                fill=True, col='class', height=3.8, aspect=1.1)
g.set_axis_labels('Science Marks', 'Density')
```

Output of step 5.4:



Explanation

Same KDE through displot(). col='class' gives one panel per class and hue='gender' compares boys and girls inside each panel.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.kdeplot()`	`sns.displot(kind='kde')`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`
Shared with	`--`	`Histogram and ECDF (kind='hist'/'ecdf')`

Teaching Note
histplot(), kdeplot() and the next rugplot() all share ONE figure-level function: displot(). Students only need to remember three kind= words.

6. Rug Plot -- rugplot() vs displot(rug=True)

A rug plot draws a **small tick for every single value** along the axis -- like the threads of a rug. On its own it is simple, so it is normally ADDED to a histogram or KDE to show exactly where the real data points are.

Dataset: student_marks.csv

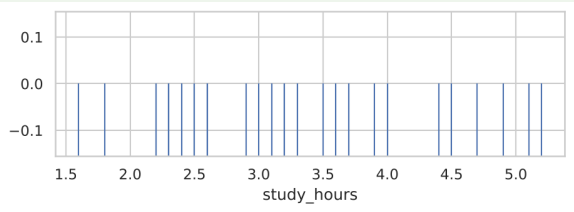
Syntax

```
sns.rugplot(data=df, x='num', hue=, height=)
sns.kdeplot(...)  +  sns.rugplot(...)   # same axes
sns.displot(data=df, x='num', kind='kde', rug=True)
```

6.1 Axes-level: a plain rug plot

```
plt.figure(figsize=(7, 2))
sns.rugplot(data=marks, x='study_hours', height=0.5)
```

Output of step 6.1:



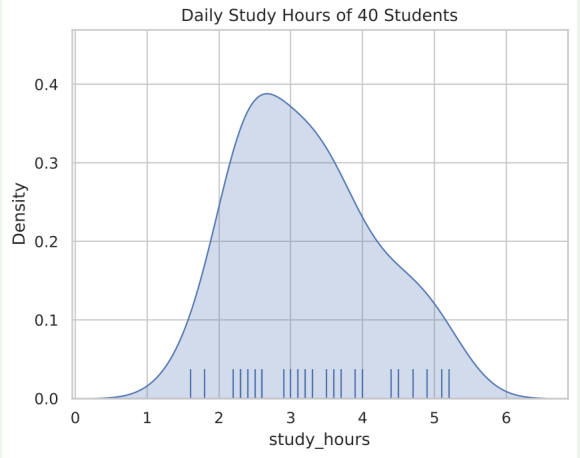
Explanation

Each tick is one student's study hours. A short, wide figure suits a rug, and height=0.5 makes the ticks half the plot height. Where ticks are crowded, many students study that long.

6.2 Combining kdeplot() + rugplot() on one axes

```
sns.kdeplot(data=marks, x='study_hours', fill=True)
sns.rugplot(data=marks, x='study_hours', height=0.08)
plt.title('Daily Study Hours of 40 Students')
```

Output of step 6.2:



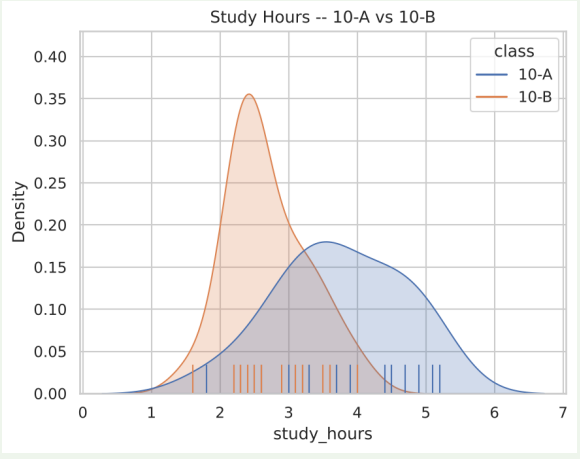
Explanation

Two axes-level functions in the same cell draw on the SAME axes. The curve shows the overall shape; the rug shows the 40 real values underneath it.

6.3 hue= -- colour the ticks by class

```
sns.kdeplot(data=marks, x='study_hours', hue='class', fill=True)
sns.rugplot(data=marks, x='study_hours', hue='class', height=0.08)
plt.title('Study Hours -- 10-A vs 10-B')
```

Output of step 6.3:



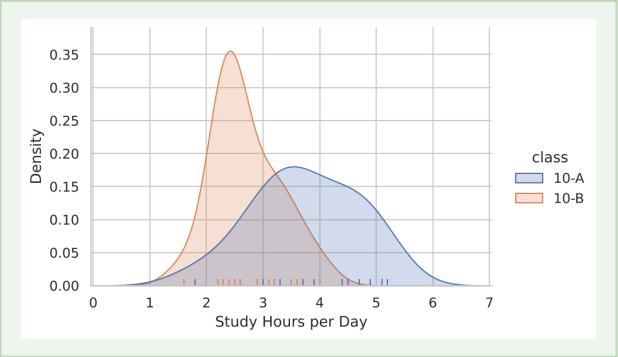
Explanation

With hue='class' on both functions, curves and ticks share the same colours. 10-A's ticks sit more to the right: they study longer.

6.4 Figure-level: displot(rug=True)

```
g = sns.displot(data=marks, x='study_hours', kind='kde',
                hue='class', fill=True, rug=True,
                height=3.8, aspect=1.5)
g.set_axis_labels('Study Hours per Day', 'Density')
```

Output of step 6.4:



Explanation

There is no separate figure-level rug function -- instead, displot() has a rug=True switch that adds the rug to any hist or kde chart. One function call replaces the two axes-level calls above.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.rugplot()`	`sns.displot(rug=True)`
Used how?	`Added on top of kdeplot()/histplot()`	`A switch inside displot()`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`

Teaching Note

Ask students: compare the rug of study_hours with the histogram of maths from Topic 3. Which students (by class) are the ones with the highest marks?

C) Matrix Plots

7. Heatmap -- heatmap() (axes-level only)

A heatmap shows a **table of numbers as colours** -- each cell gets a colour based on its value. It needs WIDE data (rows x columns), like our city x month table. Use case: spotting patterns in a grid, and showing a **correlation matrix**. Heatmap is **axes-level only** -- there is no figure-level heatmap.

Dataset: city_monthly_temperature.csv / student_marks.csv

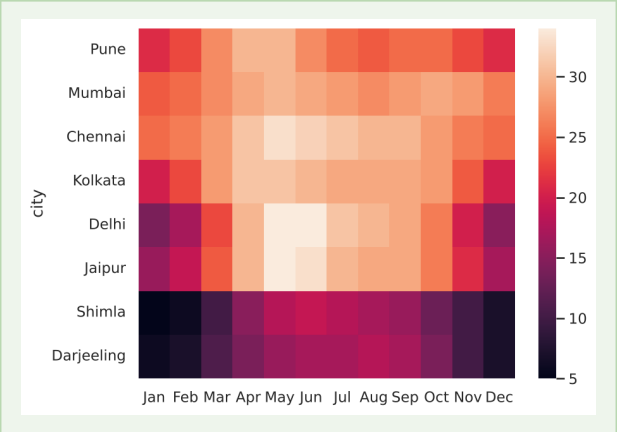
Syntax

```
sns.heatmap(data, annot=, fmt=, cmap=, linewidths=, cbar=,
            vmin=, vmax=, ax=)
df.corr()   # correlation table to feed the heatmap
```

7.1 A plain heatmap

```
sns.heatmap(city)
```

Output of step 7.1:



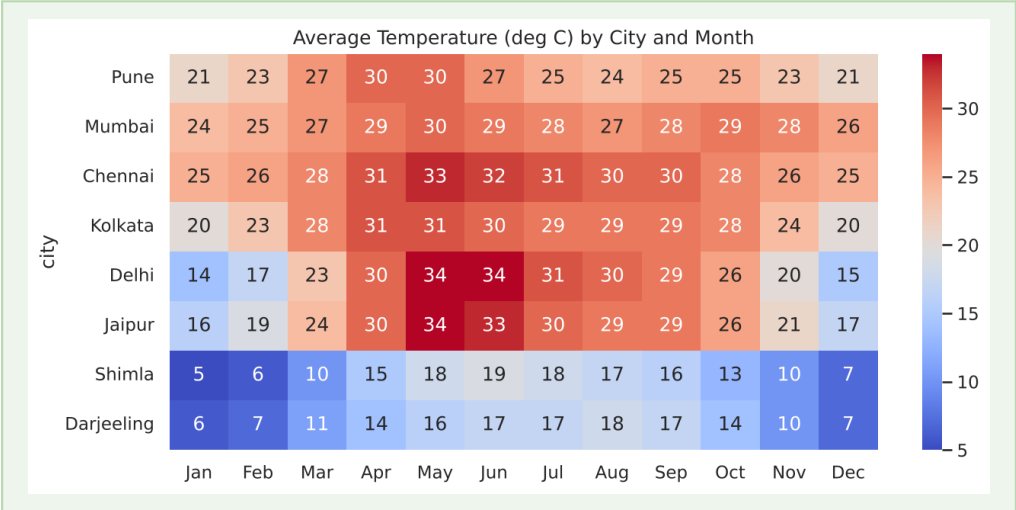
Explanation

Rows = cities, columns = months, colour = temperature. The colour bar on the right is the key. Without numbers it is still clear that Shimla and Darjeeling are the coolest rows.

7.2 annot=True, fmt= and cmap=

```
plt.figure(figsize=(10, 4.5))
sns.heatmap(city, annot=True, fmt='d', cmap='coolwarm')
plt.title('Average Temperature (deg C) by City and Month')
```

Output of step 7.2:



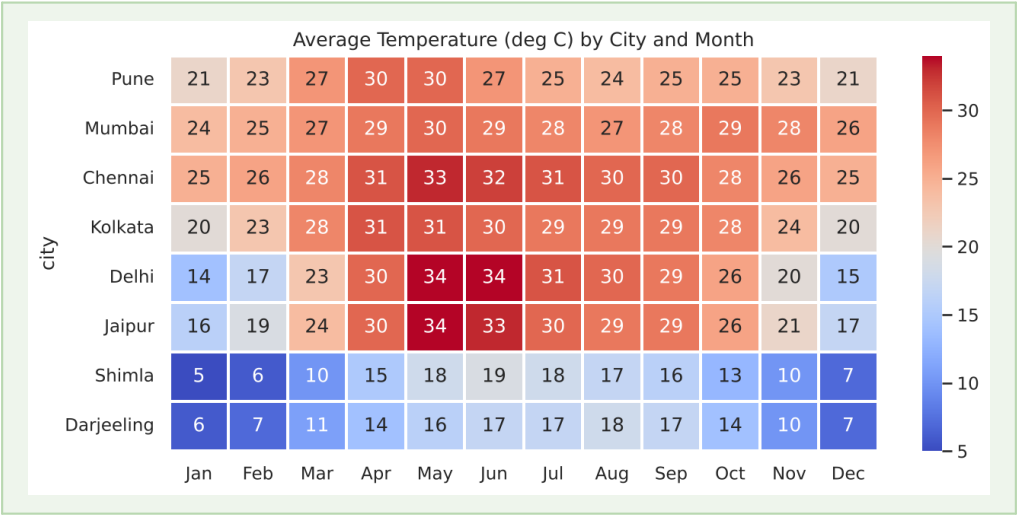
Explanation

annot=True writes the value inside every cell; fmt='d' shows whole numbers. cmap='coolwarm' uses blue for cold and red for hot -- perfect for temperatures. Delhi in May-June is the reddest cell.

7.3 linewidths= and linecolor=

```
plt.figure(figsize=(10, 4.5))
sns.heatmap(city, annot=True, fmt='d', cmap='coolwarm',
            linewidths=1, linecolor='white')
plt.title('Average Temperature (deg C) by City and Month')
```

Output of step 7.3:



Explanation

linewidths=1 draws white gaps between cells, making each value easier to read when the heatmap is printed.

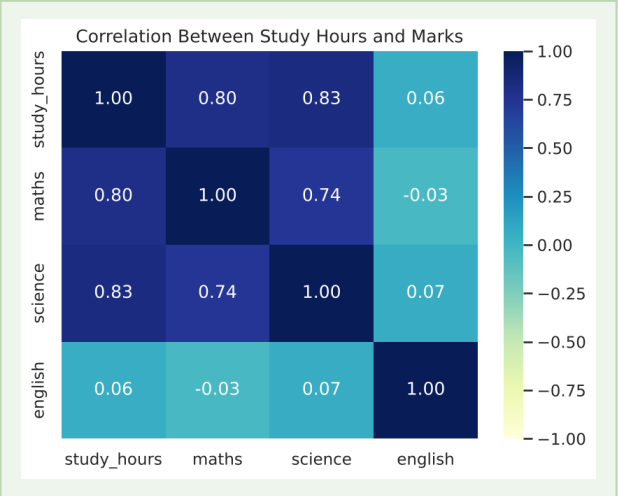
7.4 Correlation heatmap from df.corr()

```
corr = marks[['study_hours', 'maths', 'science', 'english']].corr()
print(corr.round(2))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='YlGnBu', vmin=-1, vmax=1)
plt.title('Correlation Between Study Hours and Marks')
```

Output / Result

```
             study_hours  maths  science  english
study_hours         1.00   0.80     0.83     0.06
maths               0.80   1.00     0.74    -0.03
science             0.83   0.74     1.00     0.07
english             0.06  -0.03     0.07     1.00
```

Output of step 7.4:



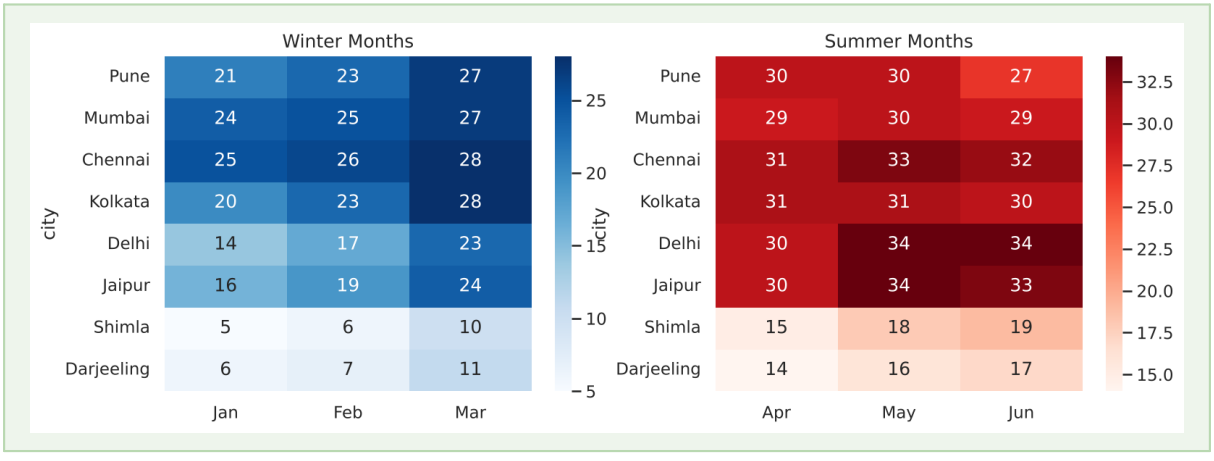
Explanation

df.corr() gives a table of correlations (-1 to +1). Study hours vs Maths is high (strong positive), while English hardly depends on study hours. vmin=-1, vmax=1 fixes the colour scale to the full correlation range.

7.5 Axes-level advantage: two heatmaps side by side

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.heatmap(city[['Jan', 'Feb', 'Mar']], annot=True, cmap='Blues',
            ax=axes[0])
sns.heatmap(city[['Apr', 'May', 'Jun']], annot=True, cmap='Reds',
            ax=axes[1])
axes[0].set_title('Winter Months')
axes[1].set_title('Summer Months')
```

Output of step 7.5:



Explanation

Because heatmap() is axes-level, it accepts ax= and fits into any plt.subplots() layout. Here winter and summer are compared side by side, each with its own colour scale.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.heatmap()`	`-- (none)`
Size	`plt.figure(figsize=(w, h))`	`--`
Panels	`plt.subplots() + ax=`	`--`
Input data	`Wide table / df.corr()`	`--`

Teaching Note
Heatmaps need numbers in a grid. Ask students to make one from the cafe data: cafe.pivot(index='weekday', columns='branch', values='total_sales_rs') -- then discuss why weekday rows repeat.

8. Cluster Map -- clustermap() (figure-level only)

A cluster map is a heatmap that **re-orders the rows and columns so that similar ones sit together**, and draws a tree (dendrogram) showing which ones are alike. Use case: finding groups -- similar customers, similar products, similar cities. clustermap() is **figure-level only**: it always builds its own figure, so it has figsize= but no ax=.

Dataset: city_monthly_temperature.csv

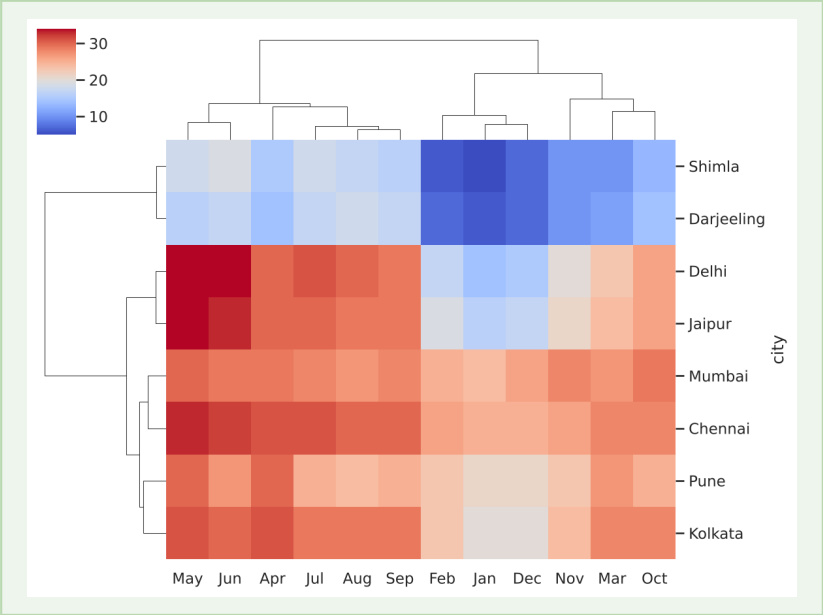
Syntax

```
sns.clustermap(data, cmap=, annot=, fmt=, figsize=,
               row_cluster=, col_cluster=, standard_scale=)
```

8.1 A plain cluster map

```
sns.clustermap(city, cmap='coolwarm', figsize=(8, 6))
```

Output of step 8.1:



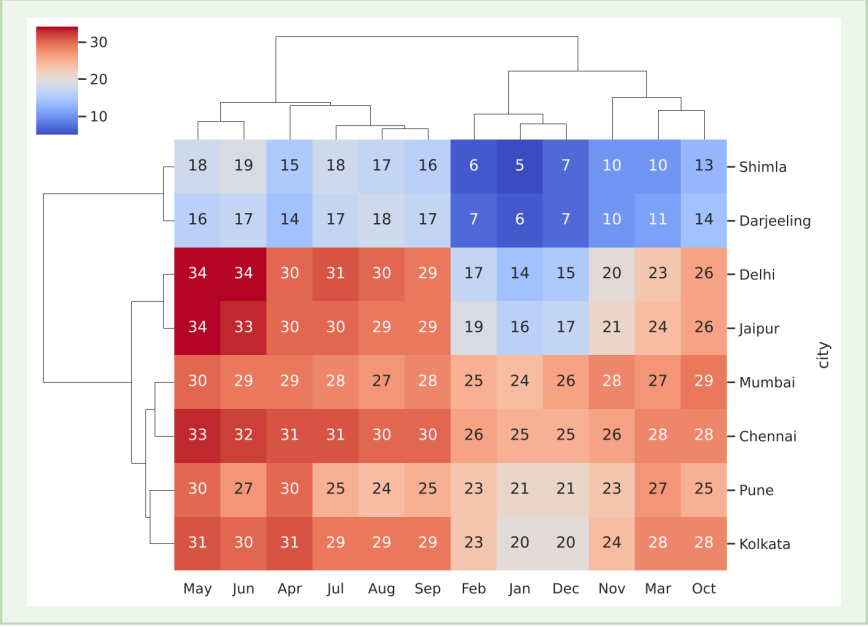
Explanation

The cities are re-ordered: Shimla and Darjeeling are pulled together (hill stations), and the hot plains form another group. The tree on the left shows how close the cities are. Note: figsize= is an argument here, because clustermap() makes its own figure.

8.2 annot=True -- show the values

```
sns.clustermap(city, cmap='coolwarm', annot=True, fmt='d',
               figsize=(9, 6.5))
```

Output of step 8.2:



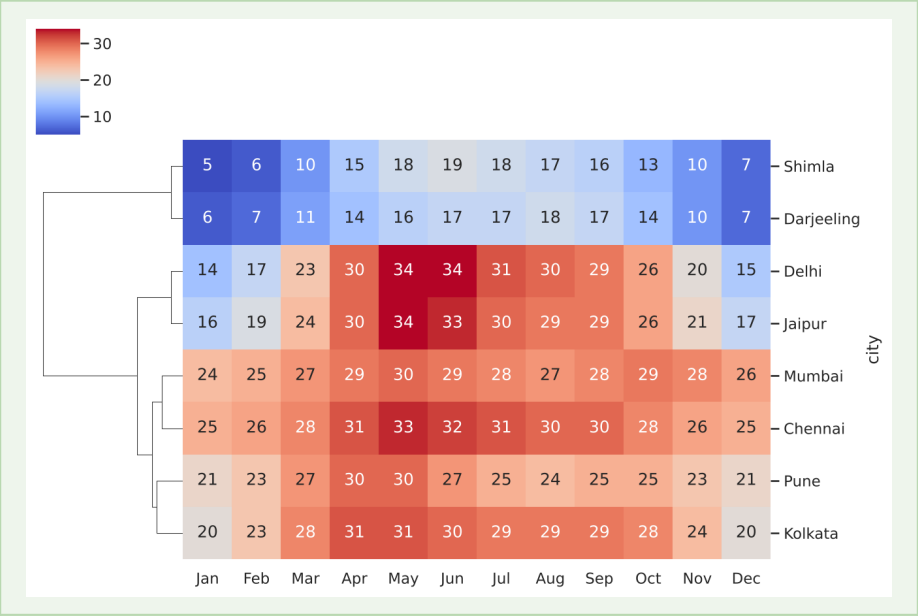
Explanation

Exactly like heatmap(), annot=True writes the temperature in every cell, so students can check why two cities were grouped together.

8.3 col_cluster=False -- keep months in calendar order

```
sns.clustermap(city, cmap='coolwarm', annot=True, fmt='d',
               col_cluster=False, figsize=(9, 6))
```

Output of step 8.3:



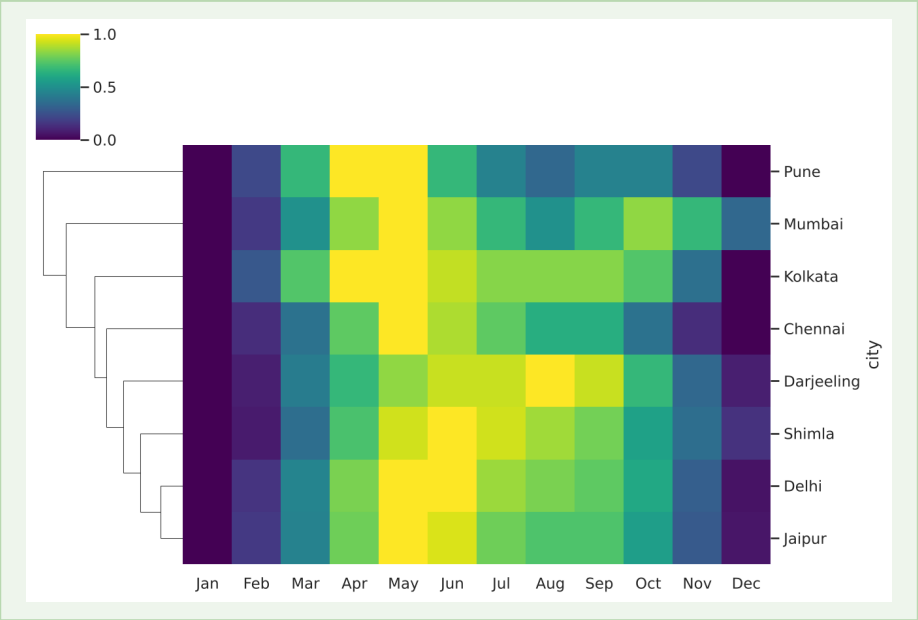
Explanation

By default the MONTHS are re-ordered too, which looks strange. col_cluster=False keeps Jan-Dec in order and only groups the cities (rows).

8.4 standard_scale=0 -- compare the pattern, not the level

```
sns.clustermap(city, cmap='viridis', col_cluster=False,
               standard_scale=0, figsize=(9, 6))
```

Output of step 8.4:



Explanation

standard_scale=0 rescales every ROW to 0-1. Now each city is compared by its SHAPE through the year: cities that peak in May (Delhi, Jaipur) group together, while Mumbai and Chennai have a flatter year.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`-- (use sns.heatmap())`	`sns.clustermap()`
Size	`--`	`figsize= argument`
Re-orders rows/cols	`No`	`Yes (with a tree)`
ax= allowed	`--`	`No -- always a new figure`

Teaching Note

Heatmap = axes-level, keeps your order. Cluster map = figure-level, finds groups for you. Ask students which two cities are the most alike, and to check it on the tree.

9. Quick Revision Cheat Sheet

Category	Chart	Axes-level	Figure-level
A) Relational	Scatter Plot	`sns.scatterplot()`	`sns.relplot(kind='scatter')`
A) Relational	Line Plot	`sns.lineplot()`	`sns.relplot(kind='line')`
B) Distribution	Histogram	`sns.histplot()`	`sns.displot(kind='hist')`
B) Distribution	KDE Plot	`sns.kdeplot()`	`sns.displot(kind='kde')`
B) Distribution	Rug Plot	`sns.rugplot()`	`sns.displot(rug=True)`
C) Matrix	Heatmap	`sns.heatmap()`	`--`
C) Matrix	Cluster Map	`--`	`sns.clustermap()`

Key Differences

Point	Axes-level	Figure-level
What it draws	One plot on one Matplotlib axes	A whole figure (FacetGrid / JointGrid / PairGrid / ClusterGrid)
Use inside plt.subplots()	Yes, with ax=	No -- always a new figure
Figure size	plt.figure(figsize=(w, h))	height= and aspect=
Split into panels	Not built in	col=, row=, col_wrap=
Legend position	Inside the plot	Outside, on the right
Returns	Matplotlib Axes	Seaborn grid object
Labels	plt.title(), plt.xlabel()	g.set_axis_labels(), g.set_titles()

Practice Exercises (try these on your own)

1. **Scatter:** plot temperature_c vs chai_cups with hue='branch'. Is the relationship positive or negative?
2. **Line:** draw cold_coffee_cups by day with relplot(kind='line', col='branch').
3. **Histogram:** histplot of english marks with hue='gender' and multiple='dodge'.
4. **KDE:** displot(kind='kde') of study_hours with col='gender'.
5. **Heatmap:** correlation heatmap of the four cafe number columns -- which pair is most strongly related?
6. **Cluster map:** clustermap of the city table with row_cluster=False -- which months are grouped together?